Nhận diện tài xế mất tập trung trên thiết bị Edge

Trần Gia Huy

Bài tập lớn môn Deep Learning

Tóm tắt nội dung

Nhận diện hành vi mất tập trung của tài xế từ camera trong xe cần chạy ngay trên thiết bị đặt trong xe (edge), nơi bộ nhớ, năng lực tính toán và điện năng đều hạn chế. Bài này so sánh bốn mạng tích chập có quy mô khác nhau (MobileNetV3-Small, EfficientNet-Lite0, ResNet18, ResNet50) trên bộ State Farm Distracted Driver, với nhãn gộp thành 6 nhóm hành vi, tập dữ liệu được chia theo tài xế và loại bỏ ảnh gần trùng. Ngoài độ chính xác (macro-F1 trên tài xế chưa từng thấy), chúng tôi đo số tham số, số phép nhân cộng (MACs), kích thước mô hình và độ trễ trên CPU trước và sau lượng tử hoá INT8 sau huấn luyện. Kết quả cho thấy mô hình nặng không chính xác hơn: EfficientNet-Lite0 (384 triệu MACs) đạt macro-F1 0,786 và 0,815 ở hai lần chạy, cao hơn ResNet50 (4,09 tỉ MACs, 0,746). Lượng tử hoá INT8 giữ nguyên độ chính xác của EfficientNet-Lite0 và ResNet18, giảm kích thước 3,5 đến 4 lần và tăng tốc 1,6 đến 3,1 lần trên CPU, nhưng làm MobileNetV3-Small sụp đổ (macro-F1 từ 0,744 xuống 0,06–0,27) ở mọi cấu hình đã thử. Phân tích lỗi cho thấy lớp `talk_passenger` chủ yếu bị nhầm với nhóm với tay/chải chuốt chứ không phải với lái xe bình thường.

1 Giới thiệu

Mất tập trung khi lái xe (dùng điện thoại, ăn uống, quay sang hành khách...) là hành vi cần được phát hiện và cảnh báo kịp thời. Một camera đặt trong xe kết hợp với mô hình phân loại ảnh có thể làm việc này. Có hai cách triển khai mô hình: gửi ảnh lên máy chủ để suy luận (cloud AI) hoặc chạy mô hình ngay trên thiết bị trong xe (edge AI). Edge AI phù hợp với bài toán này vì cảnh báo không phải chờ đường truyền, video có mặt tài xế không phải gửi đi, và hệ thống vẫn chạy khi xe mất sóng. Cái giá phải trả là thiết bị edge có bộ nhớ, năng lực tính toán và điện năng hạn chế, nên mô hình phải nhỏ và nhanh.

Việc thu nhỏ mô hình để chạy trên edge (chọn kiến trúc nhẹ, lượng tử hoá số thực 32 bit thành số nguyên 8 bit) có thể làm giảm độ chính xác. Vì vậy câu hỏi nghiên cứu của bài là: *khi bị giới hạn tài nguyên như thiết bị edge, ta đánh đổi bao nhiêu độ chính xác trên tài xế chưa từng thấy, có cách nào thu hẹp khoảng cách đó, và mô hình nào thực sự khả thi để đưa lên edge?*

Đóng góp của bài:

- Một giao thức đánh giá cố định trên State Farm: chia theo tài xế để tránh rò rỉ dữ liệu, loại bỏ ảnh gần trùng, dùng macro-F1 làm thước đo chính.
- So sánh bốn kiến trúc từ 1,5 triệu đến 23,5 triệu tham số về độ chính xác, MACs, kích thước, độ trễ và FPS, trước và sau lượng tử hoá INT8.
- Chỉ ra rằng khả năng chịu lượng tử hoá, chứ không chỉ số tham số, quyết định mô hình nào khả thi trên edge: MobileNetV3-Small nhanh nhất ở FP32 nhưng không chịu được INT8 sau huấn luyện.
- Phân tích lỗi theo lớp và theo tài xế, cho thấy giới hạn chính nằm ở lớp `talk_passenger`.

2 Công trình liên quan

Dữ liệu. Bộ State Farm Distracted Driver Detection [1] gồm ảnh chụp từ camera đặt cạnh tài xế, gán 10 lớp hành vi. Vì ảnh được trích từ video, nếu chia train/test ngẫu nhiên theo ảnh thì ảnh của cùng một tài xế (thậm chí các khung hình liền nhau) xuất hiện ở cả hai tập và điểm số cao hơn thực tế. Bài này chia theo tài xế.

Mạng nhẹ. ResNet [2] dùng kết nối tắt để huấn luyện mạng sâu. MobileNetV3 [3] kết hợp tích chập tách theo chiều sâu, khối dư đảo ngược, khối squeeze-and-excitation (SE) [4] và hàm kích hoạt hard-swish, được thiết kế bằng tìm kiếm kiến trúc. EfficientNet [5] mở rộng đồng thời chiều sâu, chiều rộng và độ phân giải; biến thể EfficientNet-Lite [6] bỏ SE và thay swish bằng ReLU6 để dễ lượng tử hoá hơn.

Lượng tử hoá. Jacob và cộng sự [7] đề xuất lượng tử hoá affine để suy luận hoàn toàn bằng số nguyên. Các tổng quan [8, 9] phân biệt lượng tử hoá sau huấn luyện (post-training quantization, PTQ) và lượng tử hoá trong huấn luyện (quantization-aware training, QAT), và lưu ý rằng các mạng dùng tích chập theo chiều sâu nhạy với PTQ.

3 Phương pháp

3.1 Dữ liệu và tiền xử lý

Chúng tôi dùng 22.424 ảnh có nhãn của thư mục huấn luyện State Farm (26 tài xế, ảnh RGB 640×480). Thư mục test của Kaggle (79.726 ảnh) không có nhãn nên không dùng.

Gộp nhãn. 10 lớp gốc được gộp thành 6 nhóm có ý nghĩa cho cảnh báo (Bảng 1). Nhóm `phone` gộp bốn lớp điện thoại nên chiếm 41,6% ảnh huấn luyện; tỉ lệ giữa lớp lớn nhất và nhỏ nhất là 4,94. Mười lớp gốc thì gần cân bằng (1.911–2.489 ảnh mỗi lớp).

Chia theo tài xế. Ảnh được trích từ video nên ảnh của cùng một tài xế rất giống nhau. Nếu chia theo ảnh, mô hình có thể nhận ra người thay vì hành vi. Chúng

Bảng 1: Gộp 10 lớp gốc thành 6 nhóm hành vi.

Nhóm	Lớp gốc State Farm
`normal`	c0 lái xe an toàn
`phone`	c1–c4 nhắn tin, gọi điện (tay trái/phải)
`device_ops`	c5 thao tác radio
`eat_drink`	c6 uống nước
`reach_grooming`	c7 với ra sau, c8 chải tóc/trang điểm
`talk_passenger`	c9 nói chuyện với hành khách



(a) Mười lớp gốc c0–c9 của cùng một tài xế.



(b) Hai cặp ảnh gần trùng (sai khác 0,5 và 1,5); chỉ giữ một ảnh mỗi cặp.

Hình 1: Dữ liệu State Farm.

tôi chia theo tài xế với seed 42: 18 tài xế huấn luyện, 4 tài xế kiểm định (val) và 4 tài xế kiểm tra (test); mỗi tài xế chỉ thuộc một tập.

Loại bỏ ảnh gần trùng. Trong mỗi nhóm (tài xế, lớp), ảnh được xét theo thứ tự tên tệp (gần với thứ tự khung hình). Mỗi ảnh được thu về ảnh xám 32×24; một ảnh bị loại nếu sai khác tuyệt đối trung bình giữa các điểm ảnh so với một ảnh đã giữ nhỏ hơn 2 (thang 0–255). Ngưỡng được chọn bằng cách xem trực tiếp các cặp ảnh (Hình 1b). Bước này loại 2.417 ảnh (10,8%), áp dụng cho cả ba tập, còn 20.007 ảnh: 13.809 / 3.617 / 2.581 cho train / val / test. Ảnh được lưu lại ở 320×240 (giữ tỉ lệ 4:3).

3.2 Mô hình và huấn luyện

Kiến trúc. Bốn mạng được chọn để phủ một dải quy mô (Bảng 2); tất cả dùng trọng số huấn luyện sẵn trên ImageNet, thay lớp phân loại cuối bằng lớp 6 đầu ra và tinh chỉnh toàn bộ.

- **MobileNetV3-Small** [3]: mô hình mục tiêu cho edge. Dùng tích chập tách theo chiều sâu (một bộ lọc cho mỗi kênh rồi tích chập 1×1 để trộn kênh), giảm mạnh số phép tính so với tích chập thường; khối SE học trọng số cho từng kênh; hàm hard-swish $x \cdot \mathrm{ReLU6}(x+3)/6$ là xấp xỉ rẻ của swish.
- **EfficientNet-Lite0** [5, 6]: dựa trên EfficientNet-B0, bỏ khối SE và dùng ReLU6, tức các phép toán có miền giá trị bị chặn, thuận lợi cho lượng tử hoá.

Bảng 2: Quy mô các mô hình (đầu ra 6 lớp, ảnh 224×224). MACs đo bằng bộ đếm phép tính của PyTorch.

Mô hình	Tham số	MACs	Vai trò
MobileNetV3-Small	1,52 M	55 M	ứng viên edge
EfficientNet-Lite0	3,38 M	384 M	ứng viên edge
ResNet18	11,18 M	1,81 G	mốc nặng vừa
ResNet50	23,52 M	4,09 G	mốc nặng

- **ResNet18 và ResNet50** [2]: mạng dư kinh điển, làm mốc so sánh nặng vừa và nặng. ResNet18 dùng khối cơ bản hai lớp 3×3; ResNet50 dùng khối nút cổ chai 1×1–3×3–1×1.

Huấn luyện. Mọi mô hình dùng chung một giao thức: 10 epoch, batch 64, AdamW [10] với learning rate 10^{-3} và weight decay 10^{-4}, lịch học cosine [11] giảm về 0 ở epoch cuối, cross-entropy với label smoothing 0,1 [12], mixed precision trên GPU. Ảnh vào được đưa về 224×224 và chuẩn hoá theo ImageNet. Tăng cường dữ liệu gồm cắt ngẫu nhiên (80–100% diện tích, tỉ lệ 0,9–1,1), xoay ±8°, thay đổi độ sáng, tương phản và bão hoà. *Không lật ngang ảnh*, vì lật sẽ biến tay phải thành tay trái và làm sai nhãn của các lớp gốc c1–c4. Checkpoint được chọn theo macro-F1 cao nhất trên val; test chỉ được đánh giá một lần với checkpoint đó.

Thước đo. Thước đo chính là macro-F1, trung bình F1 của 6 lớp với trọng số bằng nhau:

$$\mathrm{F1}_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{macro-F1} = \frac{1}{6}\sum_{c=1}^{6} \mathrm{F1}_c, \tag{1}$$

với P_c, R_c là precision và recall của lớp c. Accuracy dễ gây hiểu nhầm khi `phone` chiếm khoảng 40% dữ liệu: luôn đoán `phone` cho accuracy 0,387 trên test nhưng macro-F1 chỉ 0,093.

3.3 Lượng tử hoá INT8 và đo tốc độ

Lượng tử hoá affine. Lượng tử hoá ánh xạ một tensor số thực x sang số nguyên 8 bit q bằng một hệ số tỉ lệ s và điểm không z [7]:

$$q = \mathrm{clamp}\big(\mathrm{round}\big(\tfrac{x}{s}\big) + z,\ q_{\min}, q_{\max}\big), \qquad \hat{x} = s\,(q - z). \tag{2}$$

Với một khoảng giá trị $[\alpha, \beta]$ cần biểu diễn, $s = (\beta - \alpha)/(q_{\max} - q_{\min})$. Mỗi số chỉ còn 256 mức thay vì số thực 32 bit, nên mô hình nhỏ đi khoảng 4 lần và có thể chạy bằng phép tính số nguyên, nhanh hơn trên phần cứng hỗ trợ. Đổi lại, mỗi giá trị bị làm tròn (sai số tối đa $s/2$) hoặc bị cắt nếu nằm ngoài $[\alpha, \beta]$.

Cách làm. Mô hình PyTorch được xuất sang ONNX (opset 13) rồi lượng tử hoá bằng ONNX Runtime. Cấu hình mặc định là lượng tử hoá *tĩnh* sau huấn luyện: trọng số INT8 với một hệ số cho mỗi kênh đầu ra (per-channel), activation UINT8 với một hệ số cho mỗi tensor, định dạng QDQ (chèn nút Quantize/Dequantize quanh toán tử). Khoảng giá trị của activation được đo trên 200 ảnh huấn luyện (calibration). Chúng tôi so sánh thêm các biến thể: cách chọn khoảng giá trị *minmax* (nhỏ nhất đến lớn nhất), *entropy* (cực tiểu hoá phân kỳ KL giữa phân phối gốc và phân phối đã lượng

tử hoá) và *percentile* (cắt bỏ đuôi phân phối); 600 thay vì 200 ảnh calibration; định dạng toán tử lượng tử hoá sẵn (QOperator); và lượng tử hoá *động* (chỉ trọng số là INT8, activation được lượng tử hoá lúc chạy, không cần calibration).

Vì sao lượng tử hoá có thể làm giảm độ chính xác. (i) Sai số làm tròn ở từng lớp nhỏ nhưng cộng dồn qua hàng chục lớp. (ii) Một vài giá trị ngoại lai làm khoảng $[\alpha, \beta]$ rộng ra, khiến s lớn và phần lớn giá trị thông thường chỉ còn rất ít mức biểu diễn. (iii) Trong tích chập theo chiều sâu, mỗi kênh có bộ lọc riêng nên khoảng giá trị khác nhau nhiều giữa các kênh; một hệ số chung cho cả tensor làm mất thông tin ở các kênh nhỏ [8]. (iv) Các phép toán như hard-swish và nhánh SE (nhân activation với một hệ số trong khoảng 0–1) tạo ra phân phối activation khó biểu diễn bằng 8 bit; đây là lý do EfficientNet-Lite bỏ SE và dùng ReLU6 [6]. Do đó, sau mỗi bước chuyển đổi phải đo lại độ chính xác.

Đo tốc độ và cổng khả thi. Độ trễ đo bằng ONNX Runtime trên CPU Intel Core i5-12400F, 1 luồng, batch 1: 10 lần chạy làm nóng rồi lấy trung vị và phân vị 95 của 100 lần chạy; FPS = 1000/độ trễ trung vị (ms). Đây là phép xấp xỉ để xếp hạng tương đối, không phải số đo trên chip edge. Một mô hình được coi là *qua cổng khả thi* nếu bản INT8 giảm không quá khoảng 2 điểm macro-F1 so với bản FP32.

4 Thí nghiệm và kết quả

Môi trường. Huấn luyện chạy trên GPU của Kaggle Notebooks (lần chạy 1) và Google Colab, GPU T4 (lần chạy lại), với PyTorch và timm (cho EfficientNet-Lite0). Xuất ONNX, lượng tử hoá và đo tốc độ chạy trên CPU máy cá nhân với ONNX Runtime 1.31. Mã nguồn và nhật ký từng lần chạy nằm trong kho mã của dự án.

4.1 So sánh mô hình

Bảng 3 cho kết quả trên tập test (4 tài xế chưa từng thấy). Mọi mô hình vượt xa mốc đoán lớp đa số (macro-F1 0,093). Ba quan sát chính:

Mô hình nặng không chính xác hơn. EfficientNet-Lite0 đạt macro-F1 cao nhất ở cả hai lần chạy (0,786 và 0,815), trong khi ResNet50, nặng hơn 10,6 lần về MACs, chỉ đạt 0,746.

Val lạc quan hơn test, nhất là với mô hình lớn. Ở lần chạy 1, chênh lệch val − test là 0,120 (MobileNetV3-Small) và 0,103 (EfficientNet-Lite0), nhưng 0,150 (ResNet18) và 0,194 (ResNet50). Val chỉ có 4 tài xế và được dùng để chọn epoch, nên điểm val không phản ánh tốt khả năng tổng quát sang tài xế mới.

Nhiễu giữa các lần chạy khoảng 0,02–0,03. Chạy lại cùng cấu hình và seed trên Colab cho điểm cao hơn lần đầu 0,018–0,029. Thứ hạng EfficientNet-Lite0 trên ResNet18 không đổi, nhưng chênh lệch dưới khoảng 0,03 giữa hai cấu hình không nên coi là có ý nghĩa.

Bảng 3: Kết quả trên test (10 epoch, seed 42). Lần 1: Kaggle; lần 2: chạy lại trên Colab. Val: macro-F1 val tốt nhất của lần 1.

	Lần 1		Lần 2	
Mô hình	Acc	Macro-F1	Macro-F1	Val
MobileNetV3-Small	0,838	0,752	–	0,872
EfficientNet-Lite0	0,854	**0,786**	**0,815**	0,889
ResNet18	0,807	0,752	0,770	0,902
ResNet50	0,807	0,746	–	0,940
Đoán lớp đa số	0,387	0,093	–	–

Bảng 4: Thay đổi một siêu tham số trên MobileNetV3-Small (test). Dòng đầu là cấu hình mặc định (lần 1).

Cấu hình	Val	Macro-F1	F1 `talk`	F1 `normal`
Mặc định	0,872	**0,752**	0,391	0,683
lr $3 \cdot 10^{-4}$	0,794	0,735	0,473	0,512
lr $3 \cdot 10^{-3}$	0,895	0,744	0,470	0,568
weight decay 10^{-2}	0,841	0,741	0,458	0,667
20 epoch	0,870	0,768	0,402	0,764

4.2 Siêu tham số và số epoch

Trên MobileNetV3-Small, mỗi lần chỉ đổi một tham số so với cấu hình mặc định (Bảng 4). Không cấu hình nào vượt cấu hình mặc định; mức giảm 0,008–0,017 nằm trong phạm vi nhiễu. Đáng chú ý, F1 của `talk_passenger` tăng khoảng 0,07 trong khi `normal` giảm, tức ranh giới giữa các lớp dịch chuyển chứ mô hình không tốt hơn. Huấn luyện 20 epoch cho macro-F1 0,768, epoch tốt nhất trên val vẫn là epoch 6, nên 10 epoch là đủ cho giao thức này.

4.3 Lượng tử hoá INT8 và tốc độ

Bảng 5 và Hình 2 so sánh bản FP32 và INT8 (lượng tử hoá tĩnh, minmax, per-channel). Bản FP32 xuất qua ONNX cho macro-F1 khớp với PyTorch (sai khác khoảng 0,001), xác nhận đường ống xuất mô hình đúng.

EfficientNet-Lite0 và ResNet18 chịu tốt INT8. Macro-F1 gần như không đổi (0,814 → 0,814; 0,771 → 0,772), kích thước giảm 3,5 và 4,0 lần, độ trễ giảm 1,6 và 3,1 lần. Biến thể percentile cho kết quả tương đương (0,816 và 0,774).

MobileNetV3-Small sụp đổ sau INT8. Macro-F1 giảm từ 0,744 xuống 0,144 với cấu hình mặc định. Bảng 6 cho thấy không biến thể nào khắc phục được: entropy và QOperator vẫn 0,144, percentile chỉ đạt 0,271 (với 200 hay 600 ảnh calibration), lượng tử hoá động còn tệ hơn (0,057) và chậm hơn FP32 gần 15 lần. Trên CPU này, bản INT8 tĩnh của MobileNetV3-Small cũng chậm hơn FP32 (435 so với 605 FPS): với một mô hình rất nhỏ, chi phí của các nút lượng tử hoá/giải lượng tử hoá có thể lớn hơn phần tiết kiệm được.

4.4 Phân tích lỗi

Với EfficientNet-Lite0 (lần 2), `phone`, `device_ops` và `normal` đạt F1 từ 0,90 đến 0,95, còn `talk_passenger` chỉ đạt 0,416: precision 0,81 nhưng recall 0,28, tức mô

Bảng 5: FP32 và INT8 trên CPU (ONNX Runtime, 1 luồng, batch 1, test 2.581 ảnh). MobileNetV3-Small đo trên một checkpoint huấn luyện với lr $3 \cdot 10^{-3}$ (cùng kiến trúc).

Mô hình		MB	ms	FPS	Macro-F1
EfficientNet-Lite0	FP32	13,47	8,16	123	0,814
	INT8	3,80	5,12	195	**0,814**
ResNet18	FP32	44,71	29,05	34	0,771
	INT8	11,30	9,25	108	**0,772**
MobileNetV3-Small	FP32	6,11	1,65	605	0,744
	INT8	1,88	2,30	435	0,144

Bảng 6: Các cách lượng tử hoá MobileNetV3-Small (FP32: macro-F1 0,744; 605 FPS).

Cách lượng tử hoá	MB	FPS	Macro-F1
Tĩnh, minmax (mặc định)	1,88	435	0,144
Tĩnh, entropy	1,88	462	0,144
Tĩnh, percentile	1,88	478	0,271
Tĩnh, percentile, 600 ảnh	1,88	426	0,271
Tĩnh, QOperator	1,73	478	0,144
Động (chỉ trọng số)	1,70	41	0,057

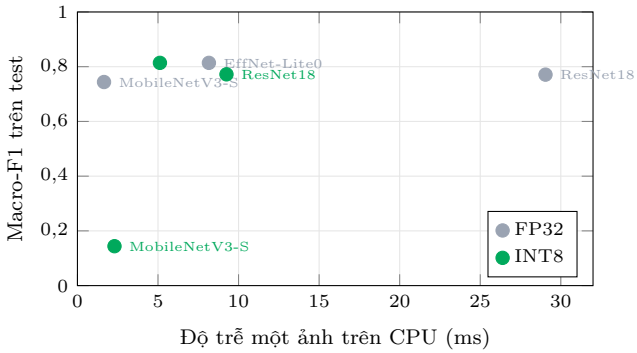
hình hiếm khi đoán lớp này. Hình 3 cho thấy 56% ảnh `talk_passenger` bị đoán là `reach_grooming`, chỉ 7% bị đoán là `normal`; giả thuyết ban đầu của chúng tôi (nhầm với `normal`) là sai. Xem trực tiếp các ảnh bị nhầm (Hình 4), nhiều ảnh mang nhãn nói chuyện với hành khách có tay giơ lên (chỉnh gương chiếu hậu) hoặc tay của hành khách chen vào khung hình, nên rất giống nhóm với tay/chải chuốt. Lớp c9 vì vậy đa dạng về hình ảnh và có phần nhập nhằng nhãn. Với ResNet18, `talk_passenger` bị nhầm sang `reach_grooming` (33%) và `phone` (28%). Nhầm lẫn đáng kể khác là `eat_drink` → `phone` (17%), có thể vì tay cầm một vật gần mặt.

Điểm cũng khác nhiều giữa các tài xế test: với EfficientNet-Lite0, accuracy từ 0,813 (p050) đến 0,953 (p035); với ResNet18, macro-F1 trên tài xế p081 chỉ 0,55. Với chỉ 4 tài xế test, việc tài xế nào rơi vào tập test ảnh hưởng đáng kể đến điểm tổng.

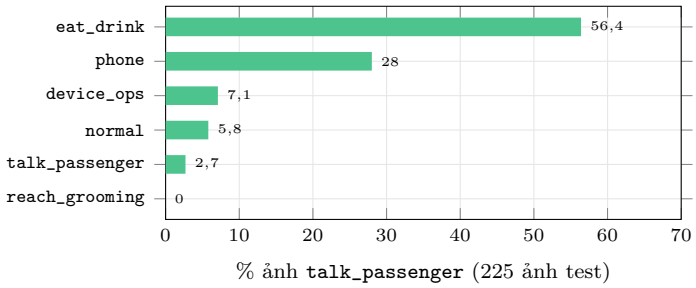
5 Thảo luận

Trả lời câu hỏi nghiên cứu. Trong các mô hình đã thử, chuyển sang mô hình nhẹ *không* làm mất độ chính xác trên tài xế mới: EfficientNet-Lite0 chính xác nhất dù chỉ cần 384 triệu MACs. Điểm đánh đổi thực sự nằm ở bước lượng tử hoá: với cùng một phương pháp INT8 sau huấn luyện, EfficientNet-Lite0 và ResNet18 không mất điểm, còn MobileNetV3-Small mất gần hết. Như vậy số tham số và MACs không đủ để chọn mô hình cho edge; khả năng chịu lượng tử hoá cũng quyết định. Với tiêu chí của bài, EfficientNet-Lite0 INT8 là lựa chọn tốt nhất: macro-F1 0,814, 3,8 MB, 195 FPS trên CPU.

Vì sao MobileNetV3-Small sụp đổ. Chúng tôi chưa xác định chắc chắn nguyên nhân. Giải thích phù hợp nhất với các quan sát là kiến trúc của MobileNetV3 dùng hard-swish và khối SE (Mục 3.3), đúng các thành phần mà EfficientNet-Lite loại bỏ để dễ lượng tử hoá;



Hình 2: Độ chính xác theo độ trễ, FP32 và INT8. Góc trên bên trái là tốt nhất. EfficientNet-Lite0 INT8 (5,1 ms; 0,814) là điểm tốt nhất trong các mô hình qua được INT8; MobileNetV3-Small INT8 nhanh nhưng sụp về độ chính xác.



Hình 3: Ảnh có nhãn `talk_passenger` được EfficientNet-Lite0 đoán thành lớp nào (dòng `talk_passenger` của confusion matrix). Chỉ 28% được đoán đúng.

việc đổi cách chọn khoảng giá trị không cải thiện đáng kể cũng cho thấy vấn đề không chỉ nằm ở giá trị ngoại lai. Kiểm chứng giả thuyết này (ví dụ để một số lớp ở FP32, hoặc dùng QAT) là việc cần làm tiếp.

Thu hẹp khoảng cách. Các thay đổi siêu tham số và số epoch không cải thiện vượt mức nhiễu. Điểm yếu lớn nhất là lớp `talk_passenger`, và phân tích lỗi cho thấy nguyên nhân chủ yếu nằm ở dữ liệu (ảnh đa dạng, nhãn nhập nhằng với nhóm với tay) hơn là ở kiến trúc, vì mọi mô hình đều yếu ở lớp này. Các thí nghiệm xử lý mất cân bằng (class weight), tăng cường dữ liệu mạnh hơn và ảnh 320×240 giữ tỉ lệ đã được chuẩn bị nhưng chưa hoàn thành.

Hạn chế. (i) Val và test mỗi tập chỉ có 4 tài xế; chạy lại cùng cấu hình lệch 0,02–0,03, nên chỉ những chênh lệch lớn mới có ý nghĩa. (ii) Mỗi cấu hình chỉ chạy một seed, chưa dùng cross-validation theo tài xế. (iii) Tốc độ đo trên CPU máy tính, chỉ để so sánh tương đối; FPS và khả năng hỗ trợ toán tử trên chip edge thật cần đo riêng. (iv) Số đo INT8 của MobileNetV3-Small lấy từ một checkpoint huấn luyện với lr $3 \cdot 10^{-3}$; ResNet50 chưa được lượng tử hoá. (v) Chỉ dùng một bộ dữ liệu, một góc camera.

6 Kết luận

Bài này so sánh bốn mạng tích chập cho bài toán nhận diện tài xế mất tập trung trên thiết bị edge, với giao thức chia theo tài xế và loại bỏ ảnh gần trùng. Mô hình nhẹ không thua mô hình nặng, và EfficientNet-Lite0 là



Hình 4: 12 ảnh `talk_passenger` bị EfficientNet-Lite0 đoán là `reach_grooming` với độ tin cậy cao nhất (góc trên: mã tài xế và xác suất). Nhiều ảnh có tay giơ lên hoặc tay hành khách trong khung hình.

lựa chọn tốt nhất: chính xác nhất, giữ nguyên độ chính xác sau INT8, nhỏ hơn 3,5 lần và nhanh hơn 1,6 lần. MobileNetV3-Small nhanh nhất ở FP32 nhưng không chịu được lượng tử hoá INT8 sau huấn luyện, cho thấy khả năng chịu lượng tử hoá cần được đánh giá trực tiếp khi chọn mô hình cho edge. Hướng phát triển gồm: đo trên chip edge thật, lượng tử hoá trong huấn luyện (QAT) cho MobileNetV3-Small, chưng cất tri thức từ mô hình lớn sang mô hình nhỏ, đánh giá với nhiều seed và cross-validation theo tài xế, và xem lại nhãn của lớp `talk_passenger`.

Tài liệu

[1] State Farm, "State Farm Distracted Driver Detection," Kaggle competition, 2016. `https://www.kaggle.com/c/state-farm-distracted-driver-detection`

[2] K. He, X. Zhang, S. Ren, J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.

[3] A. Howard *et al.*, "Searching for MobileNetV3," in *ICCV*, 2019.

[4] J. Hu, L. Shen, G. Sun, "Squeeze-and-excitation networks," in *CVPR*, 2018.

[5] M. Tan, Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *ICML*, 2019.

[6] R. Liu, "Higher accuracy on vision models with EfficientNet-Lite," TensorFlow Blog, 2020.

[7] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *CVPR*, 2018.

[8] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv:1806.08342, 2018.

[9] M. Nagel *et al.*, "A white paper on neural network quantization," arXiv:2106.08295, 2021.

[10] I. Loshchilov, F. Hutter, "Decoupled weight decay regularization," in *ICLR*, 2019.

[11] I. Loshchilov, F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *ICLR*, 2017.

[12] C. Szegedy *et al.*, "Rethinking the Inception architecture for computer vision," in *CVPR*, 2016.